

数学实验与数学软件 课程回顾

实验3—插值

三种插值方法:

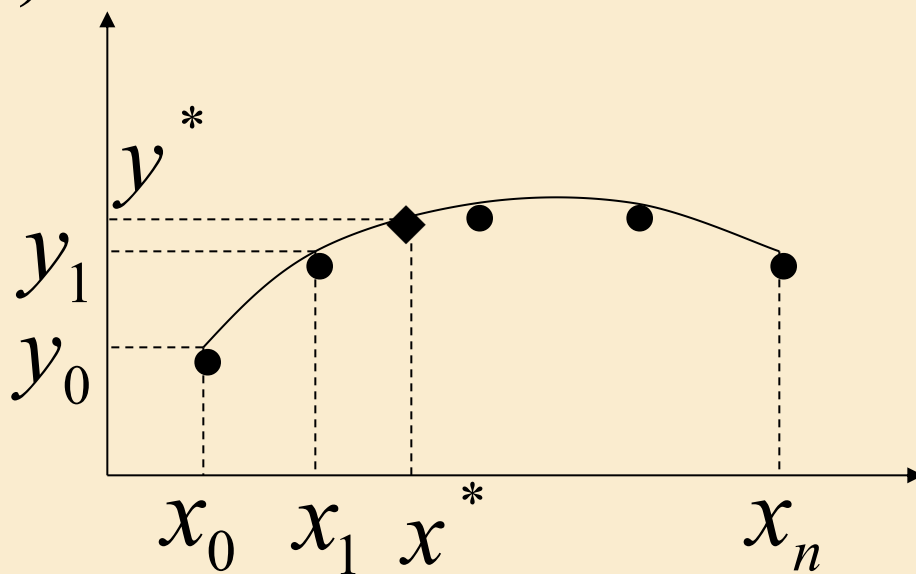
- 拉格朗日插值 (理论分析重要--得会自己写程序)
- 分段线性插值 (实用性强--得会自己写程序)
- 三次样条插值 (实用性强--得知道Matlab实现方式)

求解插值问题的基本思路

构造一个(相对简单的)函数 $y = f(x)$, 通过全部节点, 即

$$f(x_j) = y_j \quad (j = 0, 1, \dots, n)$$

再用 $f(x)$ 计算插值, 即 $y^* = f(x^*)$.



1 拉格朗日插值多项式

$$L_n(x) = \sum_{i=0}^n y_i l_i(x) \quad (3)$$

$$l_i(x) = \frac{(x-x_0)\cdots(x-x_{i-1})(x-x_{i+1})\cdots(x-x_n)}{(x_i-x_0)\cdots(x_i-x_{i-1})(x_i-x_{i+1})\cdots(x_i-x_n)}, \quad i=0,1,\cdots,n$$

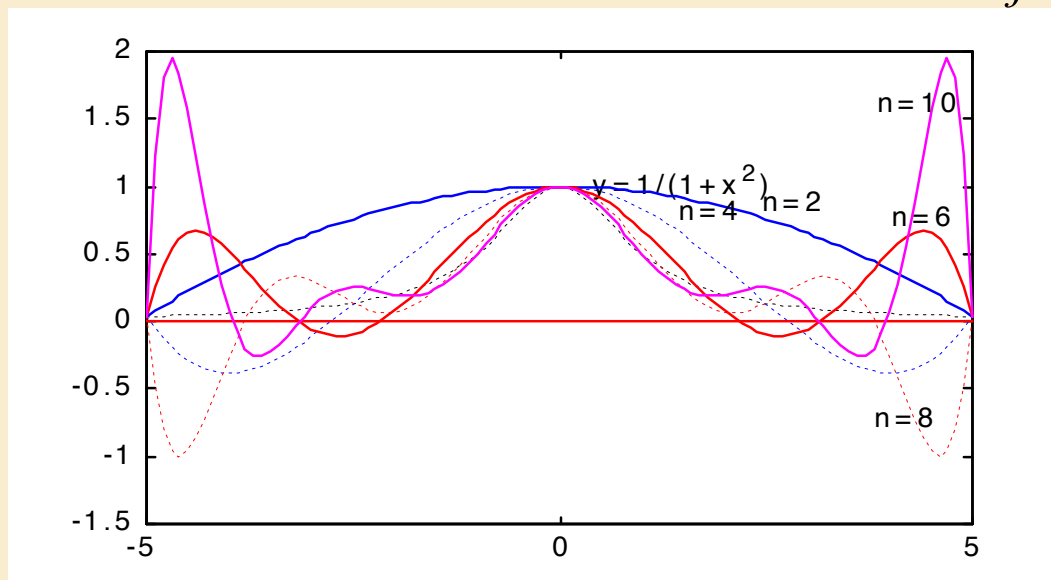
$$\therefore l_i(x_j) = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases} \quad \therefore L_n(x_j) = y_j$$

$l_i(x)$
基函数

拉格朗日插值多项式：好的误差估计，但是有龙格现象--收敛性不能保证)

$$R_n(x) = g(x) - L_n(x) = \frac{g^{(n+1)}(\xi)}{(n+1)!} \prod_{j=0}^n (x - x_j), \quad \xi \in (a, b)$$

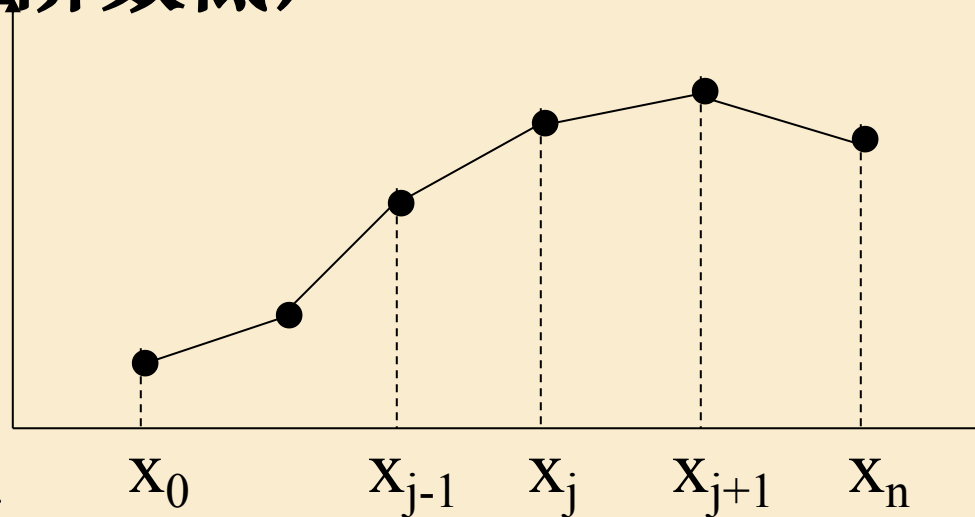
$$\left| g^{(n+1)}(\xi) \right| \leq M_{n+1} \implies \left| R_n(x) \right| \leq \frac{M_{n+1}}{(n+1)!} \prod_{j=0}^n |x - x_j|$$



2.分段线性插值（收敛但阶数低）

$$I_n(x) = \sum_{j=0}^n y_j l_j(x)$$

$$l_j(x) = \begin{cases} \frac{x - x_{j-1}}{x_j - x_{j-1}}, & x_{j-1} \leq x \leq x_j \\ \frac{x - x_{j+1}}{x_j - x_{j+1}}, & x_j \leq x \leq x_{j+1} \\ 0, & \text{其它} \end{cases}$$



$$\lim_{n \rightarrow \infty} I_n(x) = g(x), x_0 \leq x \leq x_n$$

3. 三次样条插值

$$S(x) = \{s_i(x), x \in [x_{i-1}, x_i], i = 1, \dots, n\}$$

$$1) s_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i \quad (i = 1, \dots, n)$$

$$2) S(x_i) = y_i \quad (i = 0, 1, \dots, n)$$

$$3) S(x) \in C^2[x_0, x_n]$$

4n个待定系数

$$a_i, b_i, c_i, d_i$$

$$3) \Rightarrow \begin{aligned} s_i(x_i) &= s_{i+1}(x_i), s'_i(x_i) = s'_{i+1}(x_i) \\ s''_i(x_i) &= s''_{i+1}(x_i) \end{aligned} \quad (i = 1, n-1) \quad 3')$$

2) , 3') 共 4n-2个方程

$$4) S''(x_0) = S''(x_n) = 0 \quad (\text{自然边界条件})$$

用MATLAB作插值计算

1. 拉格朗日插值:自编程序,如名为 **lagr.m** 的M文件,

第一行为 **function y=lagr(x0,y0,x)**

输入:节点 $x_0, y_0,$

插值点 x

(均为数组,长度自定义);

输出:插值 y (与 x 同长度数组))

应用时输入 x_0, y_0, x 后,

运行 **y=lagr(x0,y0,x)**

```
function y=lagr(x0,y0,x)
n=length(x0); m=length(x);
for i=1:m
    z=x(i);
    s=0;
    for k=1:n
        p=1;
        for j=1:n
            if j~=k
                p=p*(z-x0(j))/(x0(k)-x0(j));
            end
        end
        s=p*y0(k)+s;
    end
    y(i)=s;
end
```


用MATLAB作插值计算

1. 分段线性插值名为 **piecewise_linear_interp.m** 的M文件,

第一行 **function y=piecewise_linear_interp(x0,y0,x)**

输入:节点 x_0, y_0 , 插值点 x

(均为数组, 长度自定义);

输出: 插值 y (与 x 同长度数组))

应用时输入 x_0, y_0, x 后,

运行程序得到插值结果

```
%% 自编分段线性插值函数
function y_interp = piecewise_linear_interp(x, y, x_interp)
% 分段线性插值函数
% 输入:
% x - 已知数据点的 x 坐标 (1D 数组)
% y - 已知数据点的 y 坐标 (1D 数组)
% x_interp - 要插值的 x 坐标 (1D 数组)
% 输出:
% y_interp - 插值后对应的 y 坐标 (1D 数组)

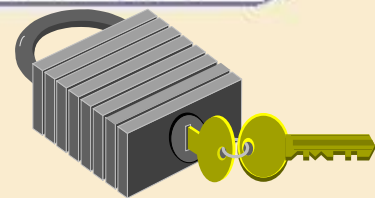
% 检查输入是否有序 --- 在Lagrange插值编程中非必需
if any(diff(x) <= 0)
    error('输入的 x 坐标必须是严格递增的。');
end

% 初始化插值结果
y_interp = zeros(size(x_interp));

% 对每个 x_interp 进行插值
for i = 1:length(x_interp)
    % 如果 x_interp 超出已知数据范围, 则采用边界值
    if x_interp(i) <= x(1)
        y_interp(i) = y(1);
    elseif x_interp(i) >= x(end)
        y_interp(i) = y(end);
    else
        % 查找 x_interp(i) 所在的区间 [x(k), x(k+1)]
        k = find(x <= x_interp(i), 1, 'last'); %

        % 线性插值计算 %
        t = (x_interp(i) - x(k)) / (x(k+1) - x(k));
        y_interp(i) = (1 - t) * y(k) + t * y(k+1);
    end
end
end
```

用MATLAB作插值计算



三次样条插值: matlab程序

$y = \text{interp1}(x_0, y_0, x, 'spline')$

或 $y = \text{spline}(x_0, y_0, x)$

实验3—积分

- 矩形公式/梯形公式/辛普森公式 (知道原理)
- 积分步长的自动选取 (了解概念, 实用性强) ,
- 高斯公式 (基于代数精度, 知道原理)
- 改进的高斯公式 (了解概念, 实用性强)

数值积分的基本思路

回忆定积分的定义

$$I = \int_a^b f(x) dx = \lim_{n \rightarrow \infty} I_n, \quad I_n = \sum_{k=1}^n f(\xi_k) \frac{b-a}{n}$$

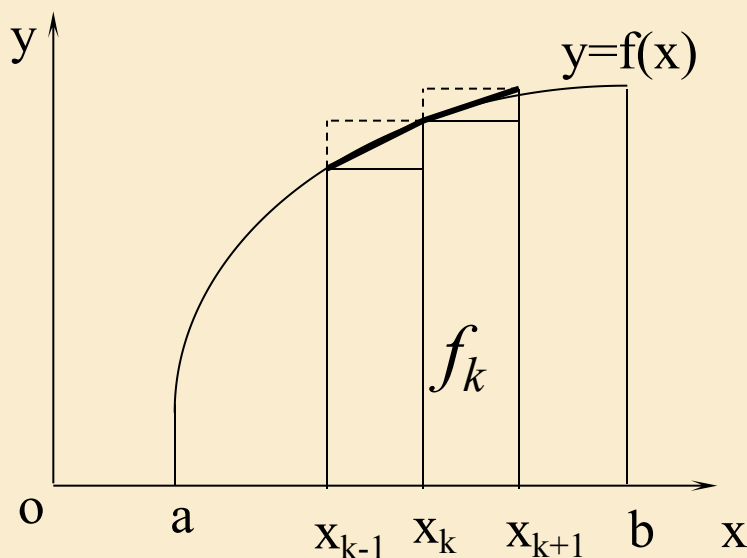
n 充分大时 I_n 就是 I 的数值积分

各种数值积分方法研究的是

ξ_k 如何取值, 区间 (a, b) 如何划分,

使得既能保证一定精度, 计算量又小。

数值积分 从矩形公式到梯形公式



$$a = x_0 < x_1 < \cdots x_k \cdots < x_n = b,$$

$$h = \frac{b-a}{n}, \quad f_k = f(x_k)$$

$$L_n = h \sum_{k=0}^{n-1} f_k \quad (1)$$

$$R_n = h \sum_{k=1}^n f_k \quad (2)$$

L_n, R_n 平均, 得到

梯形公式

$$T_n = h \sum_{k=1}^{n-1} f_k + \frac{h}{2}(f_0 + f_n) \quad (3)$$

数值积分 辛普森(Simpson)公式 (抛物线公式)

梯形公式相当于用分段线性插值函数代替 $f(x)$

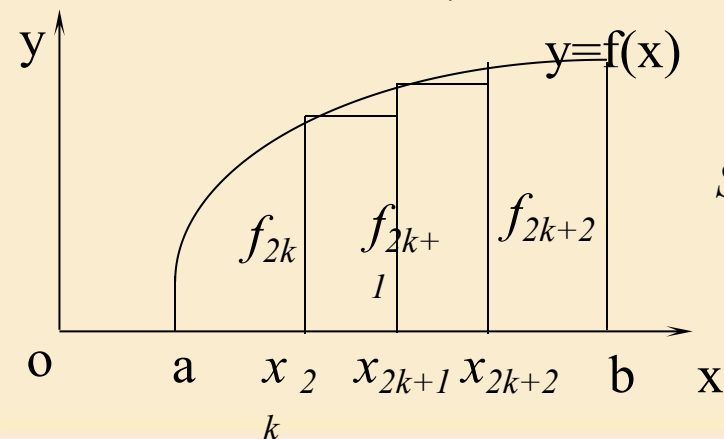


提高精度

分段二次插值函数

用 $(x_{2k}, f_{2k}), (x_{2k+1}, f_{2k+1}), (x_{2k+2}, f_{2k+2})$ 构造二次插值函数 $s_k(x)$

$$\Rightarrow \int_{x_{2k}}^{x_{2k+2}} s_k(x) dx = \frac{h}{3} (f_{2k} + 4f_{2k+1} + f_{2k+2})$$



对 k 求和(共 m 段), 得辛普森公式:

$$S_m = \frac{h}{3} (f_0 + f_{2m} + 4 \sum_{k=0}^{m-1} f_{2k+1} + 2 \sum_{k=1}^{m-1} f_{2k}), \quad h = \frac{b-a}{2m} \quad (4)$$

积分步长的自动选取

选定数值积分公式后，如何确定步长 h 以满足给定的误差 ε

梯形公式 $I - T_n \approx -\frac{h^2}{12}(f'(b) - f'(a))$ $h \rightarrow h/2$ ($n \rightarrow 2n$)

$$I - T_{2n} \approx \frac{1}{4}(I - T_n) \Rightarrow I - T_{2n} \approx \frac{1}{3}(T_{2n} - T_n)$$

用二分法只要 $|T_{2n} - T_n| \leq \varepsilon \Rightarrow |I - T_{2n}| \leq \varepsilon$

且 T_{2n} 可在 T_n 基础上计算 $T_{2n} = \frac{T_n}{2} + \frac{h}{2} \sum_{k=0}^{n-1} f_{k+1/2}$

其中 $f_{k+1/2}$ 是原分点 x_k, x_{k+1} 的中点 (记 $x_{k+1/2}$) 的函数值

高斯(Gauss)求积公式

矩形公式(1)、(2)

梯形公式(3)

辛普森公式(4)

$$I_n = \sum_{k=1}^n A_k f(x_k) \quad (7)$$

A_k 是与 f 无关的常数

代数
精度

设 $f(x) = x^k$, 用(7)计算 $I = \int_a^b f(x) dx$,

若对于 $k = 0, 1, \dots, m$ 都有 $I_n = I$,

而当 $k = m + 1$, $I_n \neq I$, 则称 I_n 的代数精度为 m .

改进的高斯公式

思路：将积分区间分小，在小区间上用不太大的 G_n 。而在节点加密一倍时能够利用原节点的函数值，需要把区间的端点作为固定节点。可以改进高斯公式

Gauss-Lobatto求积公式

$$G_n = A_1 f(a) + \sum_{k=2}^{n-1} A_k f(x_k) + A_n f(b)$$

其中 a, b 为小区间的端点, $x_2, \dots, x_{n-1}, A_1, \dots, A_n$ 为 $2n-2$ 个参数

代数精度可达到 $2n-3$

实验4—微分方程数值解

1. 实例

2. 常用的数值算法:

- (改进) 欧拉 (Euler) 方法 (非常基础)

改进的欧拉

$$\begin{cases} \bar{y}_{n+1} = y_n + hf(x_n, y_n), \\ y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, \bar{y}_{n+1})], \quad n = 0, 1, 2, \dots \end{cases}$$

- 龙格-库塔 (Runge-Kutta) 方法 (了解原理)

3. 数值算法的收敛性 (掌握原理)、稳定性与刚性方程 (了解)

4. MATLAB实现

实例 弱肉强食

模型

食饵(甲) 的密度 $x(t)$, 捕食者(乙)的密度 $y(t)$

$$\dot{x}/x = r, \quad r > 0$$

甲独立生存的增长率 r

$$\dot{x}/x = r - ay, \quad a > 0$$

乙使甲的增长率减小,
减小量与 y 成正比

$$\dot{y}/y = -d, \quad d > 0$$

乙独立生存的死亡率 d

$$\dot{y}/y = -(d - bx), \quad b > 0$$

甲使乙的死亡率减小,
减小量与 x 成正比

$$\begin{cases} \dot{x} = (r - ay)x = rx - axy \\ \dot{y} = -(d - bx)y = -dy + bxy \\ x(0) = x_0, \quad y(0) = y_0 \end{cases}$$

Volterra模型

向前欧拉公式

$$y_{n+1} = y_n + hf(x_n, y_n)$$

向后欧拉公式

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1})$$

二者平均得到梯形公式

$$y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, y_{n+1})], n = 0, 1, \dots$$

仍为隐式公式，需迭代求解

改进欧拉公式

将梯形公式的迭代过程简化为两步

$$\bar{y}_{n+1} = y_n + hf(x_n, y_n) \quad \text{预测}$$

$$y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, \bar{y}_{n+1})], n = 0, 1, \dots$$

校正

龙格-库塔方
法的一
般形式

$$\left\{ \begin{array}{l} y_{n+1} = y_n + h \sum_{i=1}^L \lambda_i k_i \\ k_1 = f(x_n, y_n) \\ k_2 = f(x_n + c_2 h, y_n + c_2 h k_1) \\ \dots \\ k_i = f(x_n + c_i h, y_n + c_i h \sum_{j=1}^{i-1} a_{ij} k_j), \quad i = 3, 4, \dots, L \end{array} \right.$$

L级?阶

$$\lambda_i, c_i, a_{ij} \text{ 满足 } \sum_{i=1}^L \lambda_i = 1, 0 \leq c_i \leq 1, \sum_{j=1}^{i-1} a_{ij} = 1$$

使精度尽量
高

常用的(经典)
龙格—库塔公式

$$\left\{ \begin{array}{l} y_{n+1} = y_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4) \\ k_1 = f(x_n, y_n) \\ k_2 = f(x_n + h/2, y_n + h k_1 / 2) \\ k_3 = f(x_n + h/2, y_n + h k_2 / 2) \\ k_4 = f(x_n + h, y_n + h k_3) \end{array} \right.$$

4级4阶

数值误差

定义 1 (整体截断误差)

从初值 $x(t_0) = x_0$ 出发, 由单步法逐步计算得 t_{n+1} 处的数值解 x_{n+1} , 则 $e_{n+1} = x(t_{n+1}) - x_{n+1}$ 称为在点 t_{n+1} 处的整体截断误差。

定义 2 (局部截断误差)

若第 n 步的数值计算没有误差, 即 $x_n = x(t_n)$, 由单步法计算出 $\tilde{x}_{n+1}$, 则 $T_{n+1} = x(t_{n+1}) - \tilde{x}_{n+1}$ 称为在点 t_{n+1} 处的局部截断误差。

定义 3 (算法精度)

如果给定的数值算法的局部截断误差为 $T_{n+1} = \mathcal{O}(h^{p+1})$, 则称该算法具有 p 阶精度。

误差分析 估计欧拉公式的局部截断误差

$y(x_{n+1})$ 在 x_n 处作Taylor展开:

$$y(x_{n+1}) = y(x_n) + hy'(x_n) + \frac{h^2}{2} y''(x_n) + O(h^3)$$

向前欧拉公式 $y_{n+1} = y_n + hf(x_n, y_n)$

$$y_n = y(x_n)$$



$$y_{n+1} = y(x_n) + hf(x_n, y(x_n)) = y(x_n) + hy'(x_n)$$

$$T_{n+1} = y(x_{n+1}) - y_{n+1} = \frac{h^2}{2} y''(x_n) + O(h^3) = O(h^2)$$

局部截断误差主项为 $\frac{h^2}{2} y''(x_n)$

误差分析

算法精度的阶的定义

一个算法的局部截断误差为 $O(h^{p+1})$



该算法具有 p 阶精度

	局部截断误差	精度
向前欧拉公式	$O(h^2)$	1阶
向后欧拉公式	$O(h^2)$	1阶
梯形公式	$O(h^3)$	2阶
改进欧拉公式	$O(h^3)$	2阶
经典龙格-库塔公式	$O(h^5)$	4阶

数值算法的收敛性和稳定性

收敛性

步长 $h \rightarrow 0$ 时数值解 y_n 无限接近解析解 $y(x_n)$

欧拉和龙格-库塔方法的共同点:只用 y_n 计算 y_{n+1} 单步法

(显式)单步法:
$$\begin{cases} y_{n+1} = y_n + h\varphi(x_n, y_n, h) \\ y_0 = y(x_0) \end{cases}$$

单步法有 p 阶精度(局部截断误差 $O(h^{p+1})$)

$$|\varphi(x, y, h) - \varphi(x, \bar{y}, h)| \leq L|y - \bar{y}| \quad (L > 0)$$

单步法收敛

整体误差 $e_n = y(x_n) - y_n = O(h^p) \rightarrow 0 \quad (h \rightarrow 0)$

向前欧拉公式 改进欧拉公式 4阶龙格-库塔公式

整体误差 $e_n = O(h)$

$e_n = O(h^2)$

$e_n = O(h^4)$

MATLAB 实现

$$\dot{x}(t) = f(t, x), x(t_0) = x_0, x = (x_1, \dots, x_n)^T, f = (f_1, \dots, f_n)^T$$

`[t,x]=ode23(@f, ts, x0)` 3级2阶龙格-库塔公式

`[t,x]=ode45(@f, ts, x0)` 5级4阶龙格-库塔公式

f是待解方程写成的函数m文件: `function dx=f(t,x)` `ts = [t0,t1, ...,tf]`
`dx=[f1; f2;...; fn];` 输出指定时刻 `ts` 的函数值

`x0`为函数初值(n维)

输出`t=ts`, `x`为相应函数值(n维)

缺省精度(相对误差 10^{-3} , 绝对误差 10^{-6}), 计算步长按精度要求自动调整.

```
function [x_vals, y_vals] = euler_method(f, xspan, y0, N) % 欧拉方法的实现
h = (xspan(2) - xspan(1))/N;
x_vals = xspan(1):h:xspan(2); % x 的值
y_vals = zeros(length(y0), N+1); % 初始化 y_vals, 列存储每个时间步的 y 值
y_vals(:, 1) = y0; % 初始条件
% 欧拉法的迭代
for n = 1:N
    y_vals(:, n+1) = y_vals(:, n) + h * f(x_vals(n), y_vals(:, n)); % 欧拉公式
end
end
```

实验5 线性代数方程组的数值解法

1. 两类数值解法 (基础) :
直接方法; 迭代方法
2. 迭代法的收敛性判断 (基础)
3. 条件数与误差分析 (了解)
4. 线性方程组数值解法的MATLAB实现

线性方程组的一般形式、两类解法

$$\begin{aligned}
 a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\
 a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\
 &\dots\dots\dots \\
 a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n
 \end{aligned}
 \quad \text{或} \quad AX=b$$

直接法 经过有限次算术运算求出精确解（实际上由于有舍入误差只能得到近似解）——— 高斯（Gauss）消元法及与它密切相关的矩阵LU分解

迭代法 从初始解出发，根据设计好的步骤用逐次求出的近似解逼近精确解 ——— 雅可比（Jacobi）迭代法和高斯—塞德尔（Gauss—Seidel）迭代法

直接法—列主元素消元法

高斯消元法条件

$$a_{kk}^{(k)} \neq 0 (k = 1, 2, \dots, n)$$

$a_{kk}^{(k)}$ (绝对值)很小时, 用它作除数
会导致舍入误差的很大增加

$$a_{k k}^{(k)} x_k + \dots + a_{k n}^{(k)} x_n = b_k^{(k)}$$

.....

$$a_{n k}^{(k)} x_k + \dots + a_{n n}^{(k)} x_n = b_n^{(k)}$$

解决
办法

选 $|a_{ik}^{(k)}| (i = k, \dots, n)$

最大的一个 (列主元)

将列主元所在行与第 k 行交换后, 再按上面的高斯消元法进行下去, 称为**列主元素消元法**。

迭代法 - 雅可比 (Jacobi) 迭代

将 A 分解为 $A = D - L - U$, 其中 $D = \text{diag}(a_{11}, a_{22}, \dots, a_{nn})$,

$$L = - \begin{bmatrix} 0 & & & \\ a_{21} & 0 & & \\ \vdots & \ddots & \ddots & \\ a_{n1} & a_{n2} & \cdots & a_{n,n-1} & 0 \end{bmatrix}, \quad U = - \begin{bmatrix} 0 & a_{12} & \cdots & a_{1n} \\ 0 & & \ddots & \vdots \\ & & \ddots & a_{n-1,1} \\ & & & 0 \end{bmatrix}$$

设对角阵 D 非奇异 (即 $a_{ii} \neq 0, i = 1, \dots, n$) $Ax = b$

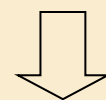
$$\Rightarrow Dx - (L + U)x = b$$



$$x = D^{-1}(L + U)x + D^{-1}b$$

$$\text{记 } B_1 = D^{-1}(L + U)$$

$$f_1 = D^{-1}b$$



迭代格式

$$x^{(k+1)} = B_1 x^{(k)} + f_1 \quad (k=0, 1, 2, \dots)$$

迭代法 - 高斯-塞德尔(Gauss-Seidel)迭代

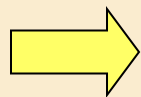
Jacobi迭代公式

$$Dx^{(k+1)} = Lx^{(k)} + Ux^{(k)} + b$$

$$x_1^{(k+1)} = -0.3x_2^{(k)} - 0.1x_3^{(k)} + 1.4$$

$$x_2^{(k+1)} = 0.2x_1^{(k)} + 0.3x_3^{(k)} + 0.5$$

$$x_3^{(k+1)} = -0.1x_1^{(k)} - 0.3x_2^{(k)} + 1.4$$



改进

$$x_1^{(k+1)} = -0.3x_2^{(k)} - 0.1x_3^{(k)} + 1.4$$

$$x_2^{(k+1)} = 0.2x_1^{(k+1)} + 0.3x_3^{(k)} + 0.5$$

$$x_3^{(k+1)} = -0.1x_1^{(k+1)} - 0.3x_2^{(k+1)} + 1.4$$

Gauss-Seidel迭代公式

$$Dx^{(k+1)} = Lx^{(k+1)} + Ux^{(k)} + b$$

在 D 非奇异的假设下 $(D-L)$ 可逆, 于是得到

$$B_2 = (D-L)^{-1}U, \quad f_2 = (D-L)^{-1}b$$

$$x^{(k+1)} = B_2x^{(k)} + f_2 \quad (k = 0, 1, 2, \dots)$$

迭代法的收敛性

Jacobi迭代

$$x^{(k+1)} = B_1 x^{(k)} + f_1$$

$$B_1 = D^{-1}(L + U)$$

$$f_1 = D^{-1}b$$

Gauss-Seidel迭代

$$x^{(k+1)} = B_2 x^{(k)} + f_2$$

$$B_2 = (D - L)^{-1}U$$

$$f_2 = (D - L)^{-1}b$$

一般迭代形式

$$x^{(k+1)} = Bx^{(k)} + f$$

原方程组的解 x^* 满足: $x^* = Bx^* + f$

迭代 k 次得到 $x^{(k)} - x^* = B^k (x^{(0)} - x^*)$

序列收敛 $x^{(k)} \rightarrow x^* (k \rightarrow \infty)$ 的充要条件

$$B^k \rightarrow 0 (k \rightarrow \infty) \Leftrightarrow B \text{ 的所有特征根 (取模) 小于 } 1$$

$$B \text{ 的谱半径 } \rho(B) = \max_{1 \leq i \leq n} |\lambda_i|$$

$\lambda_i (i = 1, \dots, n)$ 是 B 的特征根

$$\rho(B) < 1$$

迭代法的收敛性

序列收敛 $x^{(k)} \rightarrow x^* (k \rightarrow \infty)$ 的充分条件

1) 若 A 是严格对角占优的, 即 $|b_{ii}| > \sum_{j \neq i} |b_{ij}| (i = 1, \dots, n)$,

则雅可比和高斯-赛德尔迭代均收敛;

2) 若 A 对称正定, 则高斯-塞德尔迭代收敛;

3) 若 $\|B\| = q < 1$, 则迭代公式 $x^{(k+1)} = Bx^{(k)} + f$ 收敛

且 $\|x^{(k+1)} - x^*\| \leq \frac{q}{1-q} \|x^{(k+1)} - x^{(k)}\|$, q 越小收敛越快

谱半径性质: $\rho(B) \leq \|B\|$ 其中 $\|B\|$ 是任何一种矩阵范数

线性方程组数值解法的MATLAB实现

1. 求解 $Ax=b$ 用左除: $x=A\backslash b$ 。

一般切勿直接用 $\text{inv}(A)*b$

2. $[x,y]=\text{lu}(A)$, $u=\text{chol}(A)$,
 $\text{norm}(x)$, $\text{eig}(x)$,
 $\text{diag}(x)$, $\text{diag}(\text{diag}(x))$

```
%% 雅可比迭代
function [x, iter] = jacobi_iteration(A, b, x0, tol, max_iter)
% A: 系数矩阵
% b: 常数向量
% x0: 初始解
% tol: 收敛容差
% max_iter: 最大迭代次数

% 初始化
n = length(b);           % 变量个数
x = x0;                   % 初始解
D = diag(A);              % 提取对角线元素
R = A - diag(D);          % 非对角线部分

iter = 0;
while iter < max_iter
    iter = iter + 1;
    % 向量化更新公式
    x_new = (b - R * x) ./ D;
    % 检查收敛
    if norm(x_new - x, inf) < tol
        break;
    end
    % 更新解
    x = x_new;
end

if iter == max_iter
    disp('达到最大迭代次数, 未收敛');
else
    disp(['迭代次数: ', num2str(iter)]);
end
end
```

线性方程组数值解法的MATLAB实现

1. 求解 $Ax=b$ 用左除: $x=A\backslash b$

一般切勿直接用 $\text{inv}(A)*b$

%% 高斯-赛德尔迭代

function [x, iter] = gauss_seidel_iteration(A, b, x0, tol, max_iter)

% 初始化

n = length(b); % 变量个数

x = x0; % 初始解

iter = 0; % 迭代次数

% 分解 A 为下三角部分 L (包含对角线) 和上三角部分 U

L = tril(A); % 下三角部分, 含对角线

U = A - L; % 上三角部分

while iter < max_iter

iter = iter + 1;

x_old = x; % 保存上一轮的解

% 更新 x: 利用矩阵操作

x = L \ (b - U * x_old);

% 检查是否收敛

if norm(x - x_old, inf) < tol || norm(x_old - x, inf)/norm(x, inf) < tol
break;

end

end

% 检查收敛情况

if iter == max_iter

disp('达到最大迭代次数, 未收敛');

else

disp(['迭代次数: ', num2str(iter)]);

end

end

稀疏矩阵和满矩阵求解 $Ax=b$, 比较计算时间

`a=sparse(r,c,v,m,n)` (实用重要) 在第 r 行、第 c 列输入数值 v , 矩阵共 m 行 n 列, 输出 a 为稀疏矩阵, 只给出 (r,c) 及 v

`aa=full(a)` 输入稀疏矩阵 a , 输出 aa 为满矩阵 (包含零元素)

$$\text{设 } A = \begin{bmatrix} 4 & 1 & & \mathbf{0} \\ 1 & 4 & 1 & \\ & 1 & 4 & \ddots \\ \mathbf{0} & & \ddots & \ddots & 1 \\ & \mathbf{0} & & 1 & 4 \end{bmatrix}_{n \times n}$$

$$b = [1, 2, \dots, n]^T$$

```
n=500;b=[1:n]';
a1=sparse(1:n,1:n,4,n,n);
a2=sparse(2:n,1:n-1,1,n,n);
a=a1+a2+a2';
tic;x=a\b;t1=toc
aa=full(a);
tic;xx=aa\b;t2=toc
y=sum(x)
yy=sum(xx)
```

$t1, t2$ 相差巨大, 说明用稀疏矩阵计算的优点

($y=yy$ 用于简单地验证两种方法结果的一致)

实验6 非线性方程求解

1. 非线性方程 $f(x)=0$ 的数值解法:

- 迭代方法的基本原理; (基础)
- 牛顿法; 拟牛顿法. (基础, 会自己编写程序)

2. 推广到解非线性方程组 (牛顿迭代相关,
收敛性判断--基础)

3. 分岔与混沌 (了解)

非线性方程的基本解法

解方程 $f(x)=0$ 第一步——确定根的大致范围

- 图形法: 作 $f(x)$ 图形, 观察 $f(x)$ 与 x 轴的交点
- 根的隔离: 二分法

将原方程 $f(x)=0$ 改写成容易迭代的形式 $x=\varphi(x)$, 选合适的初值 x_0 , 进行迭代: $x_{k+1}=\varphi(x_k)$ ($k=0,1,2,\dots$)

局部收敛性: 只要 $\varphi(x)$ 在 x^* 的一个邻域连续且 $|\varphi'(x^*)| < 1$, 则对于该邻域内的任意初值 x_0 , 序列 $\{x_k\}$ 就收敛于 x^* 。

牛顿切线法

$f(x)$ 在 x_k 作 Taylor 展开, 去掉 2 阶及 2 阶以上项得

$$f(x) = f(x_k) + f'(x_k)(x - x_k)$$

设 $f'(x_k) \neq 0$, 用 x_{k+1} 代替右端的 x , 由 $f(x) = 0$ 得

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}, \quad \text{即令} \quad \varphi(x) = x - \frac{f(x)}{f'(x)}$$

$$\varphi'(x^*) = \frac{f(x^*)f''(x^*)}{f'(x^*)^2}, \quad \varphi''(x^*) = \frac{f''(x^*)}{f'(x^*)}$$

若 x^* 为单根

$$f(x^*) = 0, f'(x^*), f''(x^*) \neq 0$$

$$\Rightarrow \varphi'(x^*) = 0, \varphi''(x^*) \neq 0$$

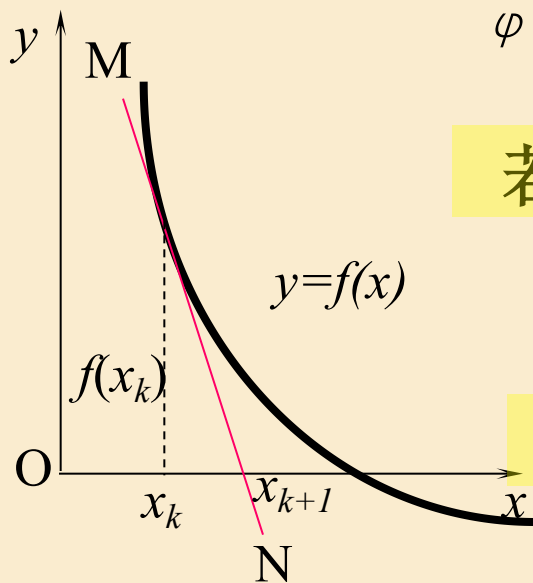
牛顿切线法2阶收敛

若 x^* 为重根

$$f(x^*) = f'(x^*) = 0 \quad \Rightarrow \quad \varphi'(x^*) \neq 0$$

牛顿切线法1阶收敛

重数越高, 收敛越慢



解非线性方程组的牛顿法

$$F(x) = 0, \quad x = [x_1, \cdots, x_n]^T, \quad F(x) = [f_1(x), \cdots, f_n(x)]^T$$

$$F'(x) = \left[\frac{\partial f_i}{\partial x_j} \right]_{n \times n} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \cdots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix} \quad \begin{array}{l} \text{解方程 } f(x)=0 \\ f(x^{k+1}) = f(x^k) + f'(x^k)(x^{k+1} - x^k) \\ x^{k+1} = x^k - \frac{f(x^k)}{f'(x^k)} \end{array}$$

$$F(x^{k+1}) = F(x^k) + F'(x^k)(x^{k+1} - x^k)$$

$$x^{k+1} = x^k - [F'(x^k)]^{-1} F(x^k)$$

$$\Rightarrow F'(x^k) \Delta x^k = -F(x^k) \quad x^{k+1} = x^k + \Delta x^k$$

拟牛顿法(Quasi-Newton)

解方程 $f(x)=0$
的牛顿割线法

$$f(x^{k+1}) = f(x^k) + a^k (x^{k+1} - x^k)$$

$$x^{k+1} = x^k - \frac{f(x^k)}{a^k} \qquad a^k = \frac{f(x^k) - f(x^{k-1})}{x^k - x^{k-1}}$$

解方程组的拟牛顿法——用 A^k 代替 $F'(x^k)$

$$\text{使 } A^k \text{ 满足 } A^k (x^k - x^{k-1}) = F(x^k) - F(x^{k-1})$$

矩阵 A^k (n^2 个未知数) 不能由这样的 n 个方程确定

用迭代方法 $A^k = A^{k-1} + \Delta A^{k-1}$ (ΔA 有不同的构造) 计算 A^k

$$\text{再求 } x^{k+1} = x^k - (A^k)^{-1} F(x^k)$$

MATLAB解非线性方程

➤ **fzero**: 单变量方程 $f(x)=0$ 求根(变号点)

一般形式 $[x, fv, ef, out] = fzero(@f, x0, opt, P1, P2, \dots)$

可选输入: “P1,P2,...”是传给f.m的参数(如果需要的话)

‘opt’是一个结构变量, 控制参数(如精度TolX)

输出: ‘fv’是函数值; ‘ef’是程序停止运行的原因(1,0,-1);

‘out’是一个结构变量, 包含:

iterations(迭代次数), funcCount (函数调用次数), algorithm(所用算法)

➤ **fsolve**: 多变量方程组 $F(x)=0$ 求解

$[x, fv, ef, out, jac] = fsolve(@f, x0, opt, P1, P2, \dots)$

一般形式

求解 $f(x)=0$ 的newton.m文件

```
function [y,z]=newton(fv,df,x0,n,tol)
x(1)=x0; b=1; k=1;
while or(k==1,abs(b)>tol*abs(x(k)))
    x(k+1)=x(k)-feval(fv,x(k))/feval(df,x(k));
    b=x(k+1)-x(k);
    k=k+1;
    if(k>n)
        error('Error: Reached maximum iteration times');break;
    end
end
y=x(k-1);
if nargout>1    fv是f(x)的函数句柄, df是f'(x)的函数句柄
    z=k-1;
end
```

参考我上课讲的牛顿法求解非线性系统代码

求解 $f(x)=0$ 的newton.m文件

```

%% 牛顿法
function [x, k] = newton_general(F, J, x0, max_iter, abs_tol, rel_tol)
% 输入参数:
% F      - 非线性方程组或函数句柄
% J      - 雅可比矩阵或导数句柄
% x0     - 初始猜测 (标量或向量)
% max_iter - 最大迭代次数
% abs_tol - 绝对误差的容差
% rel_tol - 相对误差的容差

% 初始化变量
x = x0;
k = 0;
relative_error = rel_tol + 1; % 初始化相对误差为大于rel_tol的值以进入循环

% Newton-Raphson 迭代
while (norm(feval(F, x)) > abs_tol || relative_error > rel_tol) && k < max_iter % 如果第一个操作数为 false, 直接返回 false, 不计算第二个操作数
% 计算函数值和雅可比矩阵或导数
Fx = feval(F, x);
Jx = feval(J, x);

% 更新 x 值
if length(x) == 1
% 一元情况: 直接除以导数
x_new = x - Fx / Jx;
else
% 多元情况: 使用雅可比矩阵求解
delta_x = -Jx \ Fx;
x_new = x + delta_x;
end

% 计算相对误差
relative_error = norm(x_new - x) / norm(x_new);

% 更新 x 和迭代计数
x = x_new;
k = k + 1;
end

% 检查是否达到最大迭代次数
if k >= max_iter
error('未在最大迭代次数内收敛');
end

end

```

实验7 无约束优化

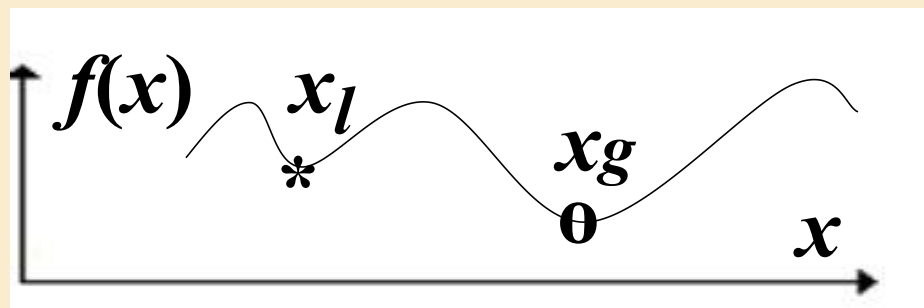
1. 优化问题的最优解条件 (基础)
2. 无约束优化的基本方法：梯度法，牛顿法，拟牛顿法 (基础，会自己编写程序)
3. (非) 线性最小二乘法
4. 优化工具箱的使用 (了解)

无约束优化: 最优解的分类和条件

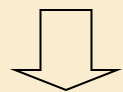
给定一个函数 $f(x)$, 寻找 x^* 使得 $f(x^*)$ 最小, 即

$$\underset{x}{\text{Min}} f(x) \quad \text{其中} \quad x = (x_1, x_2, \dots, x_n)^T \in \mathbb{R}^n$$

局部最优解



全局最优解



必要条件

$$\nabla f(x^*) = (f_{x_1}, \dots, f_{x_n})^T = 0$$

充分条件

$$\nabla f(x^*) = 0, \nabla^2 f(x^*) > 0 \quad \nabla^2 f = \left[\frac{\partial^2 f}{\partial x_i \partial x_j} \right]_{n \times n}$$

Hessian阵

求解无约束优化的基本思路

基本思想 在 $\mathbb{R}^n$ 中某一点, 确定一个搜索方向及沿该方向的移动步长, 得到使目标函数下降的新的点

迭代步骤 Step 1 初始化: 初始点 x^0 , 终止准则等

Step 2 迭代改进: 方向 d^k , 步长 α^k

$$x^{k+1} = x^k + \alpha^k d^k \quad f(x^{k+1}) < f(x^k)$$

Step 3 终止检验: 得到近优解或 $k+1 \Rightarrow k$ 转2

选择 d^k, α^k 使 f 下降更快 $\Rightarrow$ 不同算法

搜索方向的选择 最速下降法 (梯度法)

暂不考虑搜索步长,

可设 $\alpha^k=1$

将 $f(x^{k+1})$ 在 x^k 点作泰勒展开, 只保留一阶项, 有

$$f(x^{k+1}) = f(x^k + d^k) = f(x^k) + \nabla f^T(x^k) d^k$$

下降方向 $\nabla f^T(x^k) d^k < 0$

最速下降方向 $d^k = -\nabla f(x^k)$ (负梯度方向)

迭代改进格式 $x^{k+1} = x^k - \nabla f(x^k)$

算法特点 初始阶段改进较快, 最优解附近改进较慢

搜索方向的选择 Newton方法

将 $f(x^{k+1})$ 在 x^k 点作泰勒展开至二阶项, 用 d 替代 d^k

$$f(x^{k+1}) = f(x^k + d) = f(x^k) + \nabla f^T(x^k)d + \frac{1}{2}d^T \nabla^2 f(x^k)d$$

求 d 使 $f(x^{k+1})$ 极小 $\Rightarrow$ 右端对 d 导数为0 $\Rightarrow \nabla f(x^k) + \nabla^2 f(x^k)d = 0$

牛 顿 方 程 $\nabla^2 f(x^k)d = -\nabla f(x^k)$

牛 顿 方 向 $d^k = -(\nabla^2 f(x^k))^{-1} \nabla f(x^k)$

迭 代 格 式 $x^{k+1} = x^k - (\nabla^2 f(x^k))^{-1} \nabla f(x^k)$

特点 局部2阶收敛;需计算Hessian阵,它可能病态或不正定

比较 解 $F(x) = 0$ 的牛顿法 $x^{k+1} = x^k - [F'(x^k)]^{-1} F(x^k)$
 $F(x) \Leftrightarrow \nabla f(x)$

Newton方法的编程（参照牛顿法求解 非线性方程组章节）

优化工具箱

模型: $\underset{x}{\text{Min}} f(x), x \in R^n$

基本用法:

`x=fminbnd(@f,lb,ub)`

`x=fminunc(@f,x0,options,P1,P2,...)`

`x=fminsearch(@f,x0,options,P1,P2,...)`

采用分析梯度: `GradObj='on', x=fminunc(@fun,x0,opt,...)`

`fun.m`中还可以有梯度function `[f,g]=fun(x)`

`[x,resnorm,res,exitf,out,lambda,jacob]=lsqcur
vefit(@f,x0,t,y,lb,ub,options,P1,P2,...)`

`[x,resnorm,res,exitf,out,lambda,jacob]=lsqnon
lin(@fun,x0,lb,ub,options,P1,P2,...)`

实验8 线性规划

1. 基本原理和算法

- 二维线性规划的图解法
- 一般线性规划的**单纯形算法**（标准型是基础，基可行解，出基和入基）
- 一般线性规划的内点算法
- 对偶问题

2. MATLAB实现

```
[x, fval, exitflag, output, lambda] =  
linprog(c, A1, b1, A2, b2, v1, v2, x0, opt)
```

线性规划的标准型

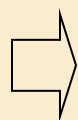
$$\max z = 3x_1 + x_2$$

$$s.t. \quad x_1 - x_2 \geq -2$$

$$x_1 - 2x_2 \leq 2$$

$$3x_1 + 2x_2 \leq 14$$

$$x_1, x_2 \geq 0$$



$$\min z = -3x_1 - x_2 + 0 \cdot x_3 + 0 \cdot x_4 + 0 \cdot x_5$$

$$s.t. \quad -x_1 + x_2 + x_3 = 2, \quad x_3 \geq 0$$

$$x_1 - 2x_2 + x_4 = 2, \quad x_4 \geq 0$$

$$3x_1 + 2x_2 + x_5 = 14, \quad x_5 \geq 0$$

$$x_1, x_2 \geq 0$$

加入松弛变量/剩余变量将不等式变为等式

标准形

$$\min \quad z = c^T x$$

$$s.t. \quad Ax = b, \quad x \geq 0$$

$$A \in R^{m \times n}, \quad m \leq n$$

实验9 非线性规划

核心方法：有效集法

基本思想：对于不等式约束的二次规划，在某可行点处将不起作用约束去掉，起作用约束视为等式约束，通过求解等式约束的二次规划来改进可行点。

若 x 为最优解,且 $\nabla g_j(x)$ ($j \in J_1$) 线性无关, 则存在

$$\nabla f(x) + \sum_{j=1}^l \lambda_j \nabla g_j(x) = 0$$

$$\lambda_1, \dots, \lambda_l \geq 0$$

~KKT条件

$$\lambda_j g_j(x) = 0, \quad j = 1, 2, \dots, l$$

互补性条件

Matlab的实现: `[x,fv,ef,out,lag,grad,hess] = fmincon(@f,x0,A1,b1,A2,b2,v1,v2,@c,opt,P1,P2,...)`

实验10 整数规划

$$\min z = -x_1 - x_2$$

$$s.t. \quad 2x_1 - x_2 \geq \frac{1}{2}$$

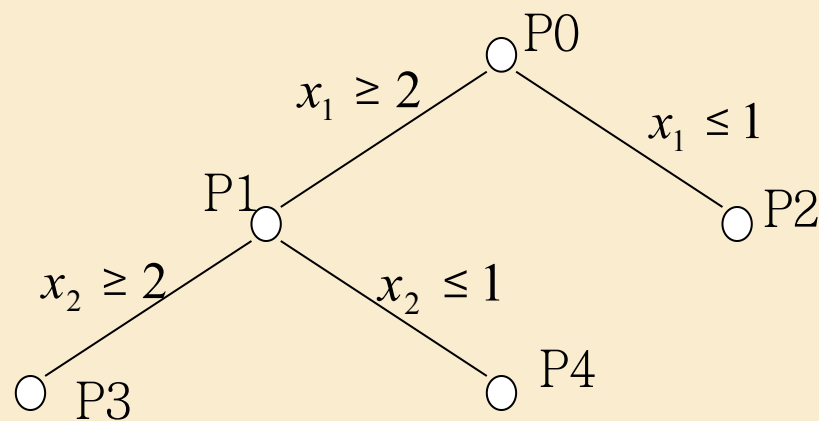
$$2x_1 + x_2 \leq \frac{11}{2}$$

$$x_2 \geq \frac{1}{2}$$

$$x_1, x_2 \in \mathbb{Z}^+$$

1、分枝定界

2、动态规划 (状态转移方程及准则函数, 资源配置问题)

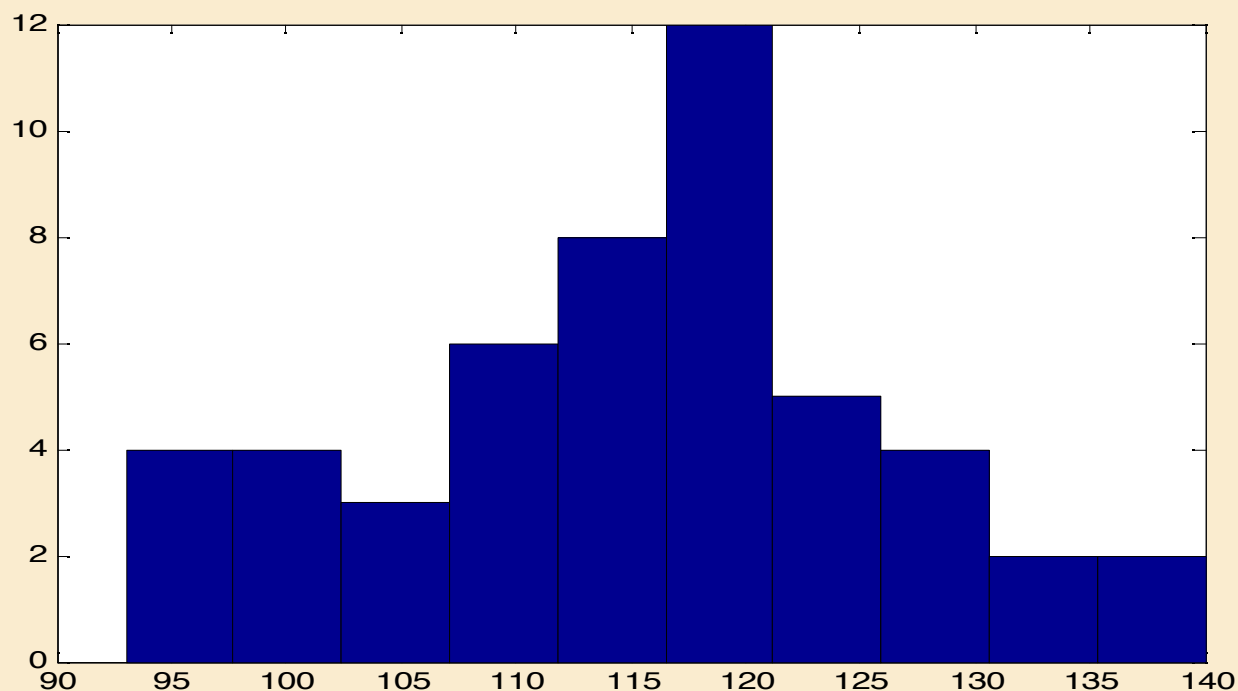


实验11 数据的统计与分析

- 1、统计基本概念（概率分布，数字特征）
- 2、用随机模拟计算数值积分

拿到数据先看分布情况

直方图(histogram), 或频数分布图 (hist/histogram)



柜台高度直方图

数据的统计与分析中的Matlab函数

命令	名称	输入	输出	注意事项
<code>[n,y]=hist(x,k)</code>	频数表	x: 原始数据行向量 k: 等分区间数	n: 频数行向量 y: 区间中点行向量	<code>[n,y]=hist(x)</code> 中 k 取缺省值 10
<code>hist(x,k)</code>	直方图	同上	直方图	同上
<code>mean(x)</code>	均值	x: 原始数据行向量		
<code>median(x)</code>	中位数	同上	中位数	
<code>range(x)</code>	极差	同上	极差	
<code>std(x)</code>	标准差	同上	标准差 s	<code>std(x,1)</code> : (3) 式中 $n-1$ 改成 n
<code>var(x)</code>	方差	同上	方差 s^2	<code>var(x,1)</code> : 同上
<code>skewness(x)</code>	偏度	同上	偏度 g_1	
<code>kurtosis(x)</code>	峰度	同上	峰度 g_2	

数据的统计与分析中的Matlab函数

分布	均匀分布	指数分布	正态分布	分布	t分布	F分布	二项分布	泊松分布
字符	unif	exp	norm	chi2	t	f	bino	poiss

功能	概率密度	分布函数	逆概率分布	均值与方差	随机数生成
字符	pdf	cdf	inv	stat	rnd

实验12 统计推断

1、参数估计 (点估计, 区间估计—方差已知/未知)

2、假设检验 (方差已知/未知)

Matlab: ttest, ttest2

假设检验

已有样本(容量 n , 均值 $\bar{x}$, 标准差 s), 要对总体均值 μ 是否等于给定值 μ_0 进行检验(假定总体服从正态分布)

假设 $H_0: \mu = \mu_0; \quad H_1: \mu \neq \mu_0$

称 H_0 为原假设(或零假设), H_1 为备选假设, 二者择其一: 接受 H_0 ; 拒绝 H_0 , 即接受 H_1 。

显著性水平 $\alpha \sim H_0$ 成立时被错误拒绝的概率。

总体方差
 σ^2 已知

$$z = \frac{\bar{x} - \mu_0}{\sigma / \sqrt{n}} \sim N(0, 1)$$

$$P(|z| \leq u_{1-\alpha/2}) = 1 - \alpha$$

$\Rightarrow$ $|z| \leq u_{1-\alpha/2}$ 时接受 H_0 ;
否则拒绝 H_0 (接受 H_1)。

称 z 检验或 u 检验

假设检验

`[h,sig,ci,zval]=ztest(x,mu,sigma,alpha,tail)`

输入参数 x 是样本 (n 维数组), μ 是 H_0 中的 μ_0 , σ 是总体标准差 σ , α 是显著性水平 α (默认时设定为 0.05), tail 是对双侧检验和两个单侧检验的标识, 用备选假设 H_1 确定: H_1 为 $\mu \neq \mu_0$ 时令 $\text{tail}=0$ (可默认); H_1 为 $\mu > \mu_0$ 时令 $\text{tail}=1$; H_1 为 $\mu < \mu_0$ 时令 $\text{tail}=-1$.

输出参数 $h=0$ 表示接受 H_0 , $h=1$ 表示拒绝 H_0 , sig 是在假设 H_0 下的概率 $P\{|Z| \geq |z|\}$, 其中 $Z \sim N(0,1)$, $z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}}$ (详见下面例 1 的说明), ci 给出 μ_0 的置信区间, zval 是样本统计量 z 的值.

实验13 回归统计分析

1、一元线性回归 —— 参数估计（点估计，区间估计）和模型有效性检验

2、多元线性回归

3、非线性回归

Matlab: regress

一元线性回归模型 $y = \beta_0 + \beta_1 x + \varepsilon$

x ~自变量 β_0, β_1 ~回归系数

ε ~随机变量(除 x 外影响 y 的随机因素的总和)

基本假设

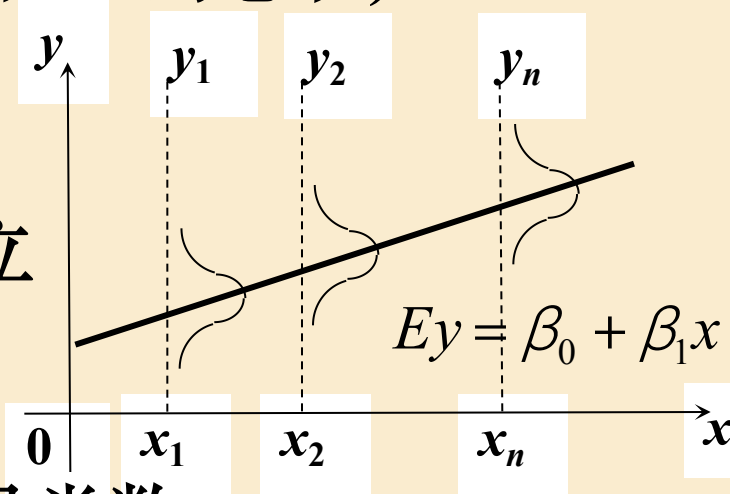
独立性: 对于不同的 x , y 相互独立

线性性: y 的期望是 x 的线性函数

齐次性: 对于不同的 x , y 的方差是常数

正态性: 对于给定的 x , y 服从正态分布

ε 是相互独立的、期望为0、方差为 σ^2 、正态分布的随机变量, 即 $\varepsilon \sim N(0, \sigma^2)$, ε 称 (随机) 误差。



回归系数的最小二乘估计

数据 $x_i, y_i (i=1, \dots, n)$ 代入 $y = \beta_0 + \beta_1 x + \varepsilon \quad \Rightarrow \quad y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$

误差平方和 $Q(\beta_0, \beta_1) = \sum_{i=1}^n \varepsilon_i^2 = \sum_{i=1}^n [y_i - (\beta_0 + \beta_1 x_i)]^2$

$$\frac{\partial Q}{\partial \beta_0} = 0, \frac{\partial Q}{\partial \beta_1} = 0 \quad \Rightarrow \quad \hat{\beta}_1 = \frac{s_{xy}}{s_{xx}}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i, \quad s_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2, \quad s_{xy} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

直线 $y = \hat{\beta}_0 + \hat{\beta}_1 x$ 通过 x_i, y_i 的均值点 $(\bar{x}, \bar{y})$

最小二乘估计

线性无偏最小方差估计

回归模型的有效性检验

$$H_0 : \beta_1 = 0, \quad H_1 : \beta_1 \neq 0$$

$$|t| = \left| \frac{\hat{\beta}_1 \sqrt{s_{xx}}}{s} \right| > t_{(n-2), 1-\alpha/2} \quad \Rightarrow \quad \text{拒绝 } H_0, \text{ 模型有效}$$

U回归平方和 $U / \sigma^2 \sim \chi_{(1)}^2$, $Q / \sigma^2 \sim \chi_{(n-2)}^2$, Q残差平方和

$$F = \frac{U}{Q/(n-2)} \sim F_{(1, n-2)}$$

$$F > F_{(1, n-2), 1-\alpha} \quad \Rightarrow \quad \text{拒绝 } H_0, \text{ 模型有效}$$

$$R^2 = U/S \quad S \text{ 偏差平方和}$$